LaTeX Indexing Editor

Name Cache SQL Queries — working with name_cache.db

1. The database

The Name Inverter keeps its results in a single SQLite file, name_cache.db, written into the installed application's own data folder rather than into any project. It has one table, cache, described in Design Overview. This document collects queries for reading it — in particular for reviewing the corrections made when the automatic suggestion was overruled.

Everything here is read-only except where a query is explicitly marked as changing the file. The database is a cache: nothing in it is unique, and it can be deleted at the cost of the lookups and corrections that would have to be made again.

Open it from a command prompt with:

```
sqlite3 -header -column "<application folder>\data\name_cache.db"
```

Paste a query at the prompt that appears, ending it with a semicolon, and press Enter. Type .quit to leave.

Every query below is followed by the output it produces. Those outputs come from the small set of illustrative rows listed in section 3, not from your own cache, so treat the shape of each result as the point rather than the names in it.

2. What shapes the queries

Six properties of the table account for most of the shape of the queries below.

- `key` is prefixed. It holds 'resolved:' followed by the name exactly as it was entered, so `substr(key, 10)` recovers the original name.
- `user_edited = 1` marks a real disagreement. A row is only written on the correction path when the final value actually differs from what was suggested, so there are no rubber-stamp rows to filter out.
- `reason` is a controlled vocabulary, not free text. It is a code chosen from a drop-down with exactly six values — `none`, `patronymic`, `particle`, `regnal`, `mismatch` and `other` — and is only ever set on a user-edited row. The drop-down defaults to `none`, so that code means 'corrected but not diagnosed' rather than 'no correction was needed'.
- `locale` is always NULL. The one place that writes a correction does not pass a locale, so no report should be built on that column.
- `created_at` is UTC text in YYYY-MM-DD HH:MM:SS form, which sorts and slices correctly as a string.

- There is no history. A second correction of the same name replaces the first, so the table holds the current decision for each name and nothing earlier.

3. The sample data

Twenty rows, standing in for a working cache part-way through a legal title. Twelve are corrections the indexer made, six were resolved automatically and accepted, and two are names the authority file had nothing for. Every example output in this document is produced from exactly these rows.

The corrections:

```
SELECT substr(key, 10) AS original_name,
       value           AS indexed_form,
       reason          AS reason_code,
       date(created_at) AS corrected_on
FROM cache
WHERE user_edited = 1
ORDER BY original_name;
```

Output

```
original_name             indexed_form               reason_code  corrected_on
------------------------  -------------------------  -----------  ------------
Albert Venn Dicey         Dicey, A. V.               none         2026-08-01
Cornelis van Bynkershoek  Bynkershoek, Cornelis van  particle     2026-07-02
Emer de Vattel            Vattel, Emer de            particle     2026-07-02
Hilary of Poitiers        Hilary of Poitiers         regnal       2026-07-06
Ibn Khaldun               Ibn Khaldun                regnal       2026-07-05
John Austin               Austin, John               mismatch     2026-07-11
John Marshall             Marshall, John             other        2026-08-01
Justinian                 Justinian I, Emperor       regnal       2026-07-06
Karl Llewellyn            Llewellyn, Karl Nickerson  none         2026-08-02
Muhammad al-Shaybani      al-Shaybani, Muhammad      particle     2026-07-03
Suharto                   Suharto                    patronymic   2026-07-08
Xiao Yang                 Xiao, Yang                 other        2026-07-09
```

Three of those rows are worth reading closely, because they show what the stored value actually is.

- `John Marshall`. The authority record offered 'Marshall, John, 1755-1835' and the rule-based fallback offered 'Marshall, John'; the indexer clicked the rule-based line to take it. A row of this shape is now historical. The date qualifier used to be removed only when the heading came back from the Library of Congress service, so a heading taken from VIAF's own record arrived with its life dates intact; every path strips them now, and this name would no longer need correcting. Corrections already recorded stay in the cache, though, so rows like this one still turn up in an established file.
- `Hilary of Poitiers`. The rules treat the last word as the surname and proposed 'Poitiers, Hilary of'. A name whose distinguishing element is a place is filed in direct order without a comma, so the indexer entered the name unchanged.
- `John Austin`. Coded mismatch: the lookup returned a different John Austin. Note that the stored heading looks like any ordinary inversion, because the table records the decision rather than the suggestion that was rejected. A wrong match is recoverable only from the reason code.

Everything else — accepted automatically, or never resolved at all:

```
SELECT substr(key, 10) AS original_name,
       value           AS indexed_form,
       date(created_at) AS resolved_on
FROM cache
WHERE user_edited = 0
ORDER BY original_name;
```

Output

```
original_name          indexed_form            resolved_on
---------------------  ----------------------  -----------
Benjamin Cardozo       Cardozo, Benjamin       2026-07-01
Hans Kelsen            Kelsen, Hans            2026-08-02
Learned Hand           Hand, Learned           2026-07-04
Oliver Wendell Holmes  Holmes, Oliver Wendell  2026-07-01
Re Smith Estate                                2026-07-10
Ronald Dworkin         Dworkin, Ronald         2026-08-01
William Blackstone     Blackstone, William     2026-07-07
interim placeholder                            2026-08-03
```

4. Classifying user corrections

4.1 Why the suggestion was overruled

The primary breakdown. A high share of mismatch is the one worth watching: it means the authority lookup is confidently returning the wrong person, which is a different problem from the rule engine mis-parsing a name.

```
SELECT
    COALESCE(reason, '(not recorded)') AS reason_code,
    CASE reason
        WHEN 'none'       THEN 'No correction needed'
        WHEN 'patronymic' THEN 'Patronymic / single-name culture'
        WHEN 'particle'   THEN 'Particle / article handling'
        WHEN 'regnal'     THEN 'Regnal or mononym'
        WHEN 'mismatch'   THEN 'Wrong person identified'
        WHEN 'other'      THEN 'Other'
        ELSE '(not recorded)'
    END      AS meaning,
    COUNT(*) AS n,
    ROUND(100.0 * COUNT(*) /
          (SELECT COUNT(*) FROM cache WHERE user_edited = 1), 1) AS pct
FROM cache
WHERE user_edited = 1
GROUP BY reason
ORDER BY n DESC;
```

Example output

```
reason_code  meaning                          n  pct
-----------  -------------------------------  -  ----
regnal       Regnal or mononym                3  25.0
particle     Particle / article handling      3  25.0
other        Other                            2  16.7
none         No correction needed             2  16.7
patronymic   Patronymic / single-name culture  1  8.3
```

```
mismatch     Wrong person identified           1  8.3
```

Read the sample result as: particle handling and regnal forms account for the bulk of the corrections, and one name was corrected because the wrong person had been matched.

4.2 Every correction, decoded

The full corpus, newest first.

```
SELECT
    substr(key, 10)                    AS original_name,
    value                              AS indexed_form,
    COALESCE(reason, '(not recorded)') AS reason_code,
    date(created_at)                   AS corrected_on
FROM cache
WHERE user_edited = 1
ORDER BY created_at DESC;
```

Example output

```
original_name             indexed_form               reason_code  corrected_on
------------------------  -------------------------  -----------  ------------
Karl Llewellyn            Llewellyn, Karl Nickerson  none         2026-08-02
Albert Venn Dicey         Dicey, A. V.               none         2026-08-01
John Marshall             Marshall, John             other        2026-08-01
John Austin               Austin, John               mismatch     2026-07-11
Xiao Yang                 Xiao, Yang                 other        2026-07-09
Suharto                   Suharto                    patronymic   2026-07-08
Justinian                 Justinian I, Emperor       regnal       2026-07-06
Hilary of Poitiers        Hilary of Poitiers         regnal       2026-07-06
Ibn Khaldun               Ibn Khaldun                regnal       2026-07-05
Muhammad al-Shaybani      al-Shaybani, Muhammad      particle     2026-07-03
Cornelis van Bynkershoek  Bynkershoek, Cornelis van  particle     2026-07-02
Emer de Vattel            Vattel, Emer de            particle     2026-07-02
```

4.3 Corrections that were never diagnosed

Rows where the drop-down was left at its default. The pattern behind these is still recoverable by eye, so they are worth a pass while the books they came from are fresh.

```
SELECT substr(key, 10) AS original_name, value AS indexed_form, created_at
FROM cache
WHERE user_edited = 1
  AND (reason IS NULL OR reason = 'none')
ORDER BY created_at DESC;
```

Example output

```
original_name      indexed_form               created_at
-----------------  -------------------------  -------------------
Karl Llewellyn     Llewellyn, Karl Nickerson  2026-08-02 09:20:00
Albert Venn Dicey  Dicey, A. V.               2026-08-01 15:02:00
```

In the sample data both are real corrections that went unlabelled: one abbreviated a name to initials, the other expanded it. Neither would be found by looking at the reason codes alone.

4.4 The shape of the corrected heading

A structural classification by comma count, independent of the reason code, so the two can be compared against each other. One comma is an ordinary inversion; no comma means the heading was filed whole, in direct order; two or more means a suffix or qualifier was carried into the heading as well.

```
WITH corrections AS (
    SELECT substr(key, 10) AS original_name, value AS indexed_form
    FROM cache WHERE user_edited = 1 AND value <> ''
)
SELECT
    CASE
        WHEN instr(indexed_form, ',') = 0 THEN 'no comma (filed whole, uninverted)'
        WHEN length(indexed_form) - length(replace(indexed_form, ',', '')) >= 2
             THEN 'two or more commas (suffix or qualifier)'
        ELSE 'one comma (Family, Given)'
    END      AS heading_shape,
    COUNT(*) AS n,
    group_concat(original_name, '; ') AS examples
FROM corrections
GROUP BY heading_shape
ORDER BY n DESC;
```

Example output

```
heading_shape                      n  examples
---------------------------------  -  --------------------------------------------------
one comma (Family, Given)          9  Emer de Vattel; Cornelis van Bynkershoek; Muha...
no comma (filed whole, uninverted)  3  Ibn Khaldun; Hilary of Poitiers; Suharto
```

The last column is shortened here to fit the page; on screen it runs on in full.

The third category is absent from this sample, and its absence is the expected result rather than a gap in the data. Generational suffixes are already handled by the rule-based inversion, so a Jr. or a III needs no correction and never reaches this table; and life dates are stripped from an authority heading before it is ever offered. A two-comma heading appearing here would therefore be worth looking at, because something unusual was filed.

4.5 Headings not filed under the trailing word

The most diagnostic query of the set. Rule-based inversion assumes the surname is the last word of the name; this isolates exactly the cases where that assumption breaks — family-name-first cultures, mononyms carrying a qualifier, and compound surnames the rules split in the wrong place.

```
WITH corrections AS (
    SELECT
        substr(key, 10) AS original_name,
        value           AS indexed_form,
        reason,
        CASE WHEN instr(value, ',') > 0
             THEN substr(value, 1, instr(value, ',') - 1)
             ELSE value
        END AS filed_under
    FROM cache
    WHERE user_edited = 1 AND value <> ''
)
```

```
SELECT
    original_name,
    filed_under,
    indexed_form,
    COALESCE(reason, '(not recorded)') AS reason_code
FROM corrections
WHERE original_name NOT LIKE '%' || filed_under
ORDER BY reason_code, original_name;
```

Example output

```
original_name  filed_under  indexed_form          reason_code
-------------  -----------  --------------------  -----------
Xiao Yang      Xiao         Xiao, Yang            other
Justinian      Justinian I  Justinian I, Emperor  regnal
```

Note what the sample result leaves out as much as what it catches. Names whose surname genuinely is the last word — Vattel, Bynkershoek, Marshall, Dicey — are absent, because the ordinary rule handled them. What remains is the set where the filing word had to be found somewhere other than the end.

Headings filed in direct order do not appear here either. Hilary of Poitiers is filed under the whole name, so the name and the filing word agree and nothing looks wrong to this test. Those cases surface instead in the no-comma row of 4.4, which is the query to read alongside this one.

4.6 Reordered, or rewritten?

Comparing letter counts either side of the correction separates a pure inversion fix from one where a regnal title was added, a middle name expanded, or a name cut back to initials. Spaces, commas and full stops are ignored, so only real changes of substance register.

```
WITH letters AS (
    SELECT
        substr(key, 10) AS original_name,
        value           AS indexed_form,
        length(replace(replace(replace(lower(substr(key,10)),
                       ' ', ''), ',', ''), '.', '')) AS src_len,
        length(replace(replace(replace(lower(value),
                       ' ', ''), ',', ''), '.', '')) AS out_len
    FROM cache
    WHERE user_edited = 1 AND value <> ''
)
SELECT
    CASE
        WHEN out_len =  src_len THEN 'reordered only'
        WHEN out_len >  src_len THEN 'expanded (name or qualifier added)'
        ELSE 'trimmed (text dropped)'
    END      AS edit_kind,
    COUNT(*) AS n,
    group_concat(original_name || ' -> ' || indexed_form, ' | ') AS examples
FROM letters
GROUP BY edit_kind
ORDER BY n DESC;
```

Example output

```
edit_kind                        n  examples
-------------------------------  -  -------------------------------------------------
reordered only                   9  Emer de Vattel -> Vattel, Emer de | Cornelis v...
```

```
expanded (name or qualifier added)  2  Justinian -> Justinian I, Emperor | Karl Llewe...
trimmed (text dropped)              1  Albert Venn Dicey -> Dicey, A. V.
```

The last column is shortened here to fit the page; on screen it runs on in full.

Reordered only is the largest group, and it includes headings where nothing actually moved: a name filed in direct order, such as Hilary of Poitiers or Suharto, has the same letters as it started with. The distinction this query draws is whether text was added or removed, not whether the words changed places.

4.7 Particle audit

Headings that begin with a particle but were filed under some other reason code, which usually means the code was picked carelessly or the case is subtler than it looks. The particle list mirrors PARTICLES in models/name_inverter.py; keep the two in step if either is extended.

```
WITH corrections AS (
    SELECT
        substr(key, 10) AS original_name,
        value           AS indexed_form,
        COALESCE(reason, '(not recorded)') AS reason_code,
        lower(CASE WHEN instr(value, ',') > 0
                   THEN substr(value, 1, instr(value, ',') - 1)
                   ELSE value END) AS filed_under
    FROM cache
    WHERE user_edited = 1 AND value <> ''
)
SELECT original_name, indexed_form, reason_code
FROM corrections
WHERE (   filed_under LIKE 'al-%'    OR filed_under LIKE 'el-%'
       OR filed_under LIKE 'van %'   OR filed_under LIKE 'von %'
       OR filed_under LIKE 'de %'    OR filed_under LIKE 'del %'
       OR filed_under LIKE 'della %' OR filed_under LIKE 'di %'
       OR filed_under LIKE 'da %'    OR filed_under LIKE 'dos %'
       OR filed_under LIKE 'le %'    OR filed_under LIKE 'la %'
       OR filed_under LIKE 'du %'    OR filed_under LIKE 'ibn %'
       OR filed_under LIKE 'bin %')
  AND reason_code <> 'particle'
ORDER BY reason_code, original_name;
```

Example output

```
original_name  indexed_form  reason_code
-------------  ------------  -----------
Ibn Khaldun    Ibn Khaldun   regnal
```

The sample data contains two particle-initial headings. al-Shaybani is correctly coded particle and so is excluded; Ibn Khaldun was coded regnal and is surfaced for a second look.

5. Orientation and upkeep

5.1 Inventory

A single line telling you what is in the file.

```
SELECT COUNT(*)                              AS total,
       SUM(user_edited = 1)                  AS corrections,
       SUM(user_edited = 0 AND value <> '') AS automatic,
       SUM(value = '')                       AS no_match,
       MIN(created_at)                       AS oldest,
       MAX(created_at)                       AS newest
FROM cache;
```

Example output

```
total  corrections  automatic  no_match  oldest               newest
-----  -----------  ---------  --------  -------------------  -------------------
20     12           6          2         2026-07-01 08:05:00  2026-08-03 08:44:00
```

5.2 Correction rate over time

How often the automatic suggestion needed overruling, by month. A rate that falls as the cache fills is the expected shape.

```
SELECT substr(created_at, 1, 7) AS month,
       COUNT(*)                 AS decisions,
       SUM(user_edited = 1)     AS corrections,
       ROUND(100.0 * SUM(user_edited = 1) / COUNT(*), 1) AS pct_corrected
FROM cache
WHERE value <> ''
GROUP BY month
ORDER BY month;
```

Example output

```
month    decisions  corrections  pct_corrected
-------  ---------  -----------  -------------
2026-07  13         9            69.2
2026-08  5          3            60.0
```

5.3 Names the authority file has nothing for

Rows with an empty value. These are re-queried on every encounter rather than being suppressed, so created_at is the most recent attempt rather than the first. A name that was never a person — a case name sent through the inverter by mistake, or a placeholder — shows up here.

```
SELECT substr(key, 10) AS original_name, created_at AS last_attempt
FROM cache
WHERE value = ''
ORDER BY created_at;
```

Example output

```
original_name        last_attempt
-------------------  -------------------
Re Smith Estate      2026-07-10 16:22:00
```

```
interim placeholder  2026-08-03 08:44:00
```

5.4 Forcing a name to be resolved afresh

The one statement in this document that changes the file. Deleting a row simply returns that name to an unresolved state, so the next encounter looks it up again.

```
DELETE FROM cache WHERE key = 'resolved:John Marshall';
```

This prints nothing when it succeeds. To confirm, run the inventory in 5.1 and check that the row count has dropped by one.

Do this with the application closed. The authority lookup is remembered for the life of the running program, so a session that is already open can still answer from memory after the row has gone.

5.5 Exporting the corrections for review

Writes the correction corpus to a file called corrections.csv in the current folder, ready to open in Excel. The three lines beginning with a dot are settings for the sqlite3 program itself rather than SQL, and take no semicolon.

```
.headers on
.mode csv
.once corrections.csv
SELECT substr(key,10)      AS original_name,
       value               AS indexed_form,
       COALESCE(reason,'') AS reason_code,
       created_at
FROM cache
WHERE user_edited = 1
ORDER BY created_at DESC;
```

The first lines of corrections.csv

```
original_name,indexed_form,reason_code,created_at
Karl Llewellyn,"Llewellyn, Karl Nickerson",none,2026-08-02 09:20:00
Albert Venn Dicey,"Dicey, A. V.",none,2026-08-01 15:02:00
John Marshall,"Marshall, John",other,2026-08-01 10:15:00
John Austin,"Austin, John",mismatch,2026-07-11 10:30:00
... (12 rows in all)
```

Then .mode column to put the display back to readable columns, or .quit to leave sqlite3 entirely.